Hands-On Lab: Terraform Basics

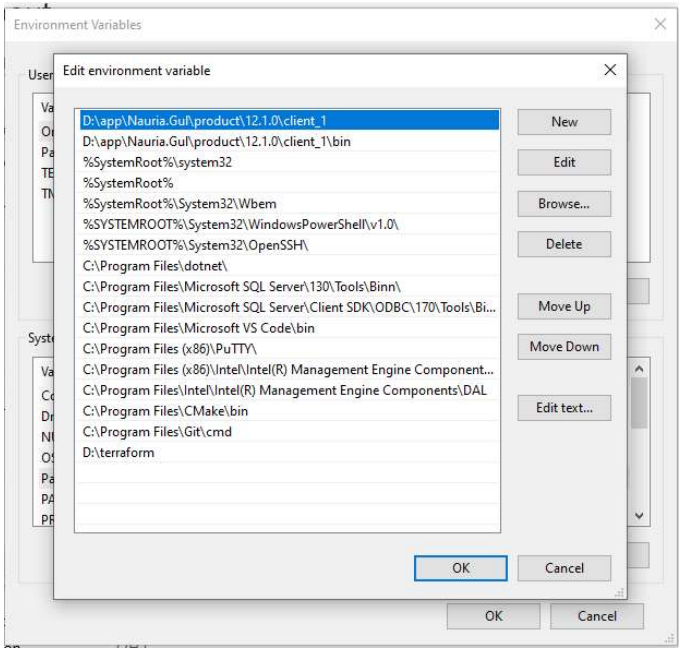


Installing Terraform on Windows

1. Download the package from the link shared below:
https://releases.hashicorp.com/terraform/1.6.5/terraform_1.6.5_windows_386.zip

2. Unzip the archive and rename it to terraform.

3. Add terraform path in Path variable as shown in below screenshot



Verify the installation

1. Verify that the installation worked by opening a new powershell or cmd session and listing Terraform's available subcommands.

2. terraform -help

```
PS D:\Fast_Workshop_Terraform> terraform -help
Usage: terraform [global options] <subcommand> [args]

The available commands for execution are listed below.
The primary workflow commands are given first, followed by
less common or more advanced commands.

Main commands:
  init          Prepare your working directory for other commands
  validate      Check whether the configuration is valid
  plan          Show changes required by the current configuration
  apply         Create or update infrastructure
  destroy       Destroy previously-created infrastructure

All other commands:
  console       Try Terraform expressions at an interactive command prompt
  fmt           Reformat your configuration in the standard style
  force-unlock  Release a stuck lock on the current workspace
  get           Install or upgrade remote Terraform modules
  graph         Generate a Graphviz graph of the steps in an operation
  import        Associate existing infrastructure with a Terraform resource
  login         Obtain and save credentials for a remote host
  logout        Remove locally-stored credentials for a remote host
  metadata      Metadata related commands
  output        Show output values from your root module
  providers     Show the providers required for this configuration
  refresh       Update the state to match remote systems
  show          Show the current state or a saved plan
  state         Advanced state management
  taint         Mark a resource instance as not fully functional
  test          Execute integration tests for Terraform modules
  untaint       Remove the 'tainted' state from a resource instance
  version       Show the current Terraform version
  workspace     Workspace management
```

Lab 1:

In this lab, you will create a Terraform configuration to deploy an Azure resource group. This resource group is the foundation for the infrastructure you will build in the subsequent exercises.

Prerequisites

- An Azure subscription.
- Terraform 0.14.9 or later
- The Azure CLI Tool installed

Install the Azure CLI tool

```
Invoke-WebRequest -Uri https://aka.ms/installazurecliwindows -OutFile .\AzureCLI.msi; Start-Process msiexec.exe -Wait -ArgumentList '/I AzureCLI.msi /quiet'; rm .\AzureCLI.msi
```



Authenticate using the Azure CLI

1. Login to az with your account credentials
   az login --user devops1@tpsonline.com --password xxxxxxxxxxx

2. Choose the account subscription id and set the account with Azure CLI:
   az account set --subscription "e15543c7-670b-435c-subscription-id“

Create a Service Principal

A Service Principal is an application within Azure Active Directory with the authentication tokens Terraform needs to perform actions on your behalf. Update the <SUBSCRIPTION_ID> with the subscription ID you specified in the previous step.

```
az ad sp create-for-rbac --name terraformrolelab --role Contributor --scopes
/subscriptions/e15543c7-670b-435c-subscription-id
```

```
{
  "appId": "e15543c7-670b-435c-95aa-a971848db8be",
  "displayName": " terraformrolelab",
  "password": “xxxxxxxxxxxxp0VwPkkeo1PCGfyEfqEhWo4aRJ",
  "tenant": “xxxxxx-xxxx-470f-8fd0-f120bc75a518"
}
```



Set your environment variables

$Env:ARM_CLIENT_ID = "<APPID_VALUE>“

$Env:ARM_CLIENT_SECRET = "<PASSWORD_VALUE>“

$Env:ARM_SUBSCRIPTION_ID = "<SUBSCRIPTION_ID>"

$Env:ARM_TENANT_ID = "<TENANT_VALUE>“

Write configuration

1. Create a provider.tf file in you folder and paste the following code:

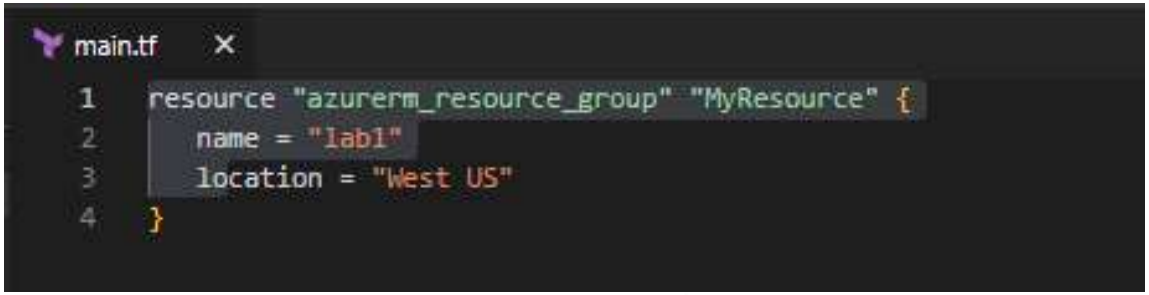
provider.tf

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0.2"
    }
  }

  required_version = ">= 1.1.0"
}

provider "azurerm" {
  features {}
}
```

2. Create a main.tf file in you folder and paste the following code:



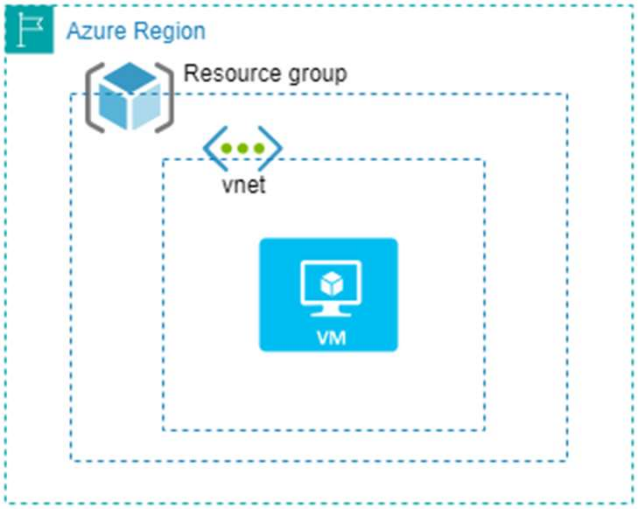
3. Run terraform init to initialize your working directory.

4. Run terraform validate to make sure your configuration is syntactically valid and internally consistent.

5. Run terraform apply to apply your configuration.

Lab 2:

In this lab, you will create a Terraform configuration to deploy an Azure resource group, Virtual network and a VM on Azure cloud.



Task:

Assign a public IP to the vm created in lab 2 with terraform code.



Thank you!

***All logos belong to their respective owners**